Confluent Cloud Terraform Deployment — User Event Enricher PTF UDF (early access example)

> This example deploys the **User Event Enricher** PTF UDF (source in examples/ptf_udf_row_driven/java/) to **Confluent Cloud** using Terraform. All infrastructure (environment, Kafka cluster, Flink compute pool, service accounts, API keys) and Flink SQL statements are declared as Terraform resources.

Table of Contents

- **1.0 Overview**
  - **1.1 How this differs from the other deployment paths**
- **2.0 How it works**
  - **2.1 Infrastructure provisioning**
  - **2.2 JAR delivery**
  - **2.3 Statement flow**
- **3.0 Prerequisites**
- **4.0 How to run**
  - **4.1 Deploy**
  - **4.2 Monitor**
  - **4.3 Tear down**
- **5.0 Resources**

1.0 Overview

1.1 How this differs from the other deployment paths

Aspect	Confluent Cloud Terraform (this example)	Confluent Platform SQL Client
Where it runs	Confluent Cloud	Confluent Platform on Minikube
How SQL is submitted	`confluent_flink_statement` Terraform resources	`sql-client.sh -f` on the JobManager pod
UDF JAR delivery	`confluent_flink_artifact` (uploaded to CC)	`kubectl exec` to Flink pods
Entry point	`make deploy-cc-ptf-udf-row-driven`	`make deploy-cp-ptf-udf-row-driven`
Requires code compilation	✅ (Java + Gradle for UDF JAR)	✅ (Java + Gradle for UDF JAR)
Statement lifecycle	Managed by Terraform state	Managed by Flink session cluster

Aspect	Confluent Cloud Terraform (this example)	Confluent Platform SQL Client
Same codebase for both?	✅ (**same Java UDF code**, different Terraform vs SQL Client for deployment)	✅ (**same Java UDF code**, different Terraform vs SQL Client for deployment)

2.0 How it works

2.1 Infrastructure provisioning

Terraform declares the full Confluent Cloud stack:

Resource	Purpose
`confluent_environment`	Isolated CC environment (`ptf-udf`)
`confluent_kafka_cluster`	Standard single-zone Kafka cluster (AWS us-east-1)
`confluent_schema_registry_cluster`	Stream Governance Essentials (auto-provisioned with environment)
`confluent_service_account`	`ptf-udf` service account with FlinkDeveloper, ResourceOwner, and Assigner roles
`confluent_flink_compute_pool`	Flink compute pool (max 10 CFU)
`confluent_api_key`	Flink-specific API key for statement submission
`flink_api_key_rotation` module	Rotating API key pair for the Flink service account

2.2 JAR delivery

On Confluent Cloud, UDF JARs are uploaded as **Flink artifacts** via the `confluent_flink_artifact` resource. The JAR is built from examples/ptf_udf_row_driven/java/ and uploaded directly from the local build output. The `CREATE FUNCTION ... USING JAR` statement references the artifact using a `confluent-artifact://` URI.

2.3 Statement flow

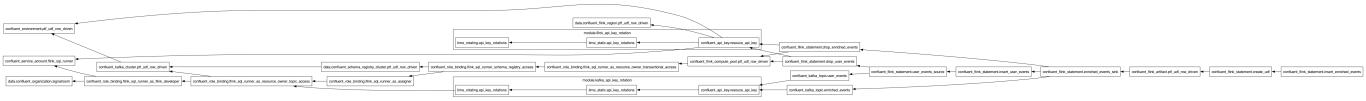
Terraform manages the SQL statements as `confluent_flink_statement` resources with explicit `depends_on` ordering:

```
┌──────────────────────────────────────────────────────────────────┐
│  Step 1:  DROP TABLE IF EXISTS user_events              → OK      │
│  Step 2:  CREATE TABLE user_events (...)                → OK      │
│  Step 3:  INSERT INTO user_events VALUES (sample data)  → submitted │
│  Step 4:  DROP TABLE IF EXISTS enriched_events          → OK      │
│  Step 5:  CREATE TABLE enriched_events (...)            → OK      │
│  Step 6:  Upload UDF JAR as confluent_flink_artifact    → OK      │
│  Step 7:  CREATE FUNCTION user_event_enricher           → OK      │
```

```
│            USING JAR 'confluent-artifact://...'                          │
│  Step 8:   INSERT INTO enriched_events                      → submitted  │
│            SELECT ... FROM TABLE(user_event_enricher())                  │
└──────────────────────────────────────────────────────────────────────────┘
```

Step 8 is a **long-running streaming job**. It runs continuously, reading from `user_events` and writing enriched output to `enriched_events`.

Once deployment completes, Terraform generates a visual **resource graph** at `examples/ptf_udf_row_driven/cc_deploy/terraform.png`, providing an at-a-glance view of the infrastructure and resource dependencies:



3.0 Prerequisites

- macOS with Homebrew or Linux with apt-get
- Java 21 and Gradle installed (for building the UDFs JAR)
- Terraform installed
- A Confluent Cloud account with a **Cloud API key** and **secret** (create one here)
- The UDF JAR must be built before deploying:

```
make build-ptf-udf-row-driven   # builds the UDF fat JAR from
examples/ptf_udf_row_driven/java/
```

4.0 How to run

All commands are run from the **project root** (where the `Makefile` lives).

4.1 Deploy

A single target builds the UDF JAR and runs `terraform apply`:

```
make deploy-cc-ptf-udf-row-driven CONFLUENT_API_KEY=<your-key>
CONFLUENT_API_SECRET=<your-secret>
```

Behind the scenes this runs:

Step	What it does
1	`./gradlew clean shadowJar` — builds the UDF fat JAR from `examples/ptf_udf_row_driven/java/`
2	`terraform init` — initializes the Terraform working directory

Step	What it does
3	`terraform apply -auto-approve` — provisions all CC infrastructure and submits Flink SQL statements
4	Generates a Terraform visualization at `docs/images/terraform-visualization.png`

4.2 Monitor

Monitor the running Flink statements in the Confluent Cloud Console:

1. Navigate to your **ptf-udf** environment
2. Open the **Flink** tab to see compute pools and running statements
3. Open the **Topics** tab to inspect `user_events` and `enriched_events`

4.3 Tear down

To destroy all Confluent Cloud resources created by Terraform:

```
make teardown-cc-ptf-udf-row-driven CONFLUENT_API_KEY=<your-key>
CONFLUENT_API_SECRET=<your-secret>
```

This runs `terraform destroy -auto-approve`, removing all Flink statements, the compute pool, Kafka cluster, service accounts, and the environment.

5.0 Resources

- Confluent Terraform Provider
- confluent_flink_statement Resource
- confluent_flink_artifact Resource
- Create a User-Defined Function with Confluent Cloud for Apache Flink